LLM Internals 計算例題集 — 目次

参考: github.com/amitshekhariitbhu/llm-internals ・ 全 24 章 / 50 問 ・ 各問題に「概要 → 問題 → 解説 → 解答」

01 トークナイズと BPE — Tokenization / Byte Pair Encoding

- 01-1 BPE のマージを手計算する
- 01-2 語彙サイズと埋め込み行列のパラメータ数
- 01-3 文字数・単語数・トークン数の換算

02 Attention の Q・K・V — Math behind Attention

- 02-1 X·W で Q, K, V を求める
- 02-2 スコア $Q \cdot K^T$ を計算する
- 02-3 softmax で重みを出し、重み·V で出力を求める

03 $\sqrt{d_k}$ によるスケーリング — Scaling Factor

- 03-1 内積の分散が d_k になることを確かめる
- 03-2 スケーリングの有無で softmax を比べる

04 Causal Mask — Causal Masking

- 04-1 マスク後の softmax を計算する
- 04-2 マスク行列の有効要素数を数える

05 クロスエントロピー損失 — Cross-Entropy Loss

- 05-1 多クラスの CE を計算する
- 05-2 二値の BCE を計算する
- 05-3 勾配 p − y を求める
- 05-4 Perplexity に変換する

06 誤差逆伝播 — Backpropagation

- 06-1 連鎖律で勾配を求める（2層の小さなネット）
- 06-2 勾配降下で重みを更新する

07 Feed-Forward Network — FFN

- 07-1 FFN のパラメータ数（4倍に広げる場合）
- 07-2 ReLU / GELU / SwiGLU の出力を計算する
- 07-3 FFN の FLOPs を数える

08 正規化 — BatchNorm / LayerNorm / RMSNorm

- 08-1 BatchNorm と LayerNorm を同じデータで比べる
- 08-2 RMSNorm を計算し LayerNorm と比べる

09 Transformer 全体 — Transformer Architecture

- 09-1 1 層あたりのパラメータ数
- 09-2 GPT-2 small の総パラメータ数を見積もる

10 KV Cache — KV Cache

- 10-1 KV Cache のメモリ量（GB）
- 10-2 KV Cache の有無で計算量を比べる

11 Paged Attention — Paged Attention

- 11-1 事前確保方式で無駄になるメモリを求める
- 11-2 必要なブロック数と内部断片化

12 Flash Attention — Flash Attention

- 12-1 Online softmax を手計算で追う
- 12-2 HBM の読み書き量を比べる

13 MHA / MQA / GQA — Grouped Query Attention

- 13-1 KV ヘッド数と KV Cache の削減率
- 13-2 GQA でのヘッドのグループ分け

14 RoPE — Rotary Position Embedding

- 14-1 2 次元回転を計算する
- 14-2 内積が相対位置だけで決まることを確かめる

15 LoRA — Low-Rank Adaptation

- 15-1 学習するパラメータの削減率
- 15-2 W + BA のマージを計算する

16 Mixture of Experts — MoE

- 16-1 ルーターの softmax と Top-k の選択
- 16-2 総パラメータ数とアクティブなパラメータ数

17 Speculative Decoding / Medusa / EAGLE — 投機的デコーディング

- 17-1 受理率 α から期待受理トークン数を求める
- 17-2 速度向上率を見積もる

18 Continuous Batching — Continuous Batching

- 18-1 Static と Continuous で GPU 利用率を比べる

19 Prefill と Decode — Inference Optimization

- 19-1 TTFT・TPOT・E2E レイテンシ
- 19-2 算術強度（compute-bound か memory-bound か）

20 Prompt Caching — Prompt Caching

- 20-1 キャッシュの書き込み・読み出しのコストを計算する

21 量子化 — Quantization

- 21-1 FP16 / INT8 / INT4 のメモリ量
- 21-2 INT8 の量子化と逆量子化、誤差

22 サンプリングと停止 — Temperature / Top-k / Stop Token

- 22-1 温度付き softmax
- 22-2 Top-k / Top-p の候補を絞り込む

23 RLHF — Reward Model / KL Penalty

- 23-1 Bradley-Terry 損失
- 23-2 KL ペナルティ付きの報酬

24 LLM ウォーターマーク — Watermarking

- 24-1 Green トークン数から z スコアを求めて判定する

例題 01-1　BPE のマージを手計算する

これは何？ 何に使う？

何か　テキストをトークンに分ける方法の 1 つ。よく一緒に出てくる文字のペアを繰り返しくっつけて、語彙（トークンの一覧）を作る。

何のため　語彙サイズを抑えながら、知らない単語も知っている部品に分解して扱うため。GPT や Llama などほとんどの LLM が BPE 系のトークナイザを使っている。

問題

次のコーパス（単語: 出現回数）で BPE（Byte Pair Encoding）を学習する。

hug	pug	pun	bun	hugs
10	5	12	4	5

初期語彙は文字 {b, g, h, n, p, s, u} とする。

(1) 最初の 3 回のマージで作られるトークンを順に答えよ。

(2) 学習後、未知語 **"bugs"** と **"mug"** はどうトークン化されるか。

解説

Step 1: 隣り合う文字ペアの出現回数を数える

各単語を文字に分け、ペアごとに「その単語の出現回数」を足し合わせる。

```
u g : hug 10 + pug 5 + hugs 5 = 20  ← 最多
p u : pun 12 + pug 5 = 17    u n : pun 12 + bun 4 = 16
h u : hug 10 + hugs 5 = 15  g s : 5    b u : 4
```

→ 1 回目のマージは **u + g → "ug"**

Step 2: "ug" を 1 つの記号として数え直す

```
h ug, p ug, p u n, b u n, h ug s
u n : 12 + 4 = 16  h ug : 15  p u : 12  p ug : 5  ug s : 5
```

→ 2 回目のマージは **u + n → "un"**

Step 3: もう一度数え直す

```
h ug, p ug, p un, b un, h ug s
h ug : 15  p un : 12  p ug : 5  ug s : 5  b un : 4
```

→ 3 回目のマージは **h + ug → "hug"**

Step 4: 新しい単語は「学習した順」にマージを適用する

```
bugs : b u g s → (u+g) b ug s → これ以上適用できない
mug  : m u g   → (u+g) m ug   → "m" は語彙にない → [UNK]
```

解答

(1) ① **ug** (20回)　② **un** (16回)　③ **hug** (15回)

(2) "bugs" → **[b, ug, s]**　/　"mug" → **[[UNK], ug]**

★ BPE は「最も頻出するペアを 1 つにまとめる」を繰り返すだけ。
実用の Byte-level BPE は 256 種のバイトを初期語彙にするので [UNK] が出ない。

例題 01-2　語彙サイズと埋め込み行列のパラメータ数

これは何？ 何に使う？

何か　トークン ID を d 次元のベクトルに変換する表（ルックアップテーブル）。
出力側の LM Head は逆に、ベクトルを語彙ごとのスコアに戻す。

何のため　ただの番号である ID を、ニューラルネットが計算できる数値ベクトルにするため。
意味の近い単語ほど近いベクトルになるよう学習される。

問題

GPT-2 small は語彙サイズ V = 50,257、埋め込み次元 d = 768 である。
(1) トークン埋め込み行列のパラメータ数はいくつか。
(2) 出力層（LM Head: d → V の線形層、バイアスなし）を埋め込みと
　　共有しない場合、両方を合わせたパラメータ数はいくつか。
(3) (1) の行列を FP16（1 パラメータ 2 バイト）で保存すると何 MB か。

解説

埋め込み行列は「語彙数 × 次元」の表

トークン ID ごとに d 次元のベクトルを 1 行ずつ持つので、形は V × d。

```
50,257 × 768 = 38,597,376  ≈ 38.6 M
```

LM Head は d → V の行列（d × V）

形は転置だが要素数は同じ。共有しなければ 2 倍になる。

```
38,597,376 × 2 = 77,194,752  ≈ 77.2 M
```

GPT-2 は weight tying（共有）を使うので、実際は (1) の分だけで済む。

メモリ量 = パラメータ数 × バイト数

```
38,597,376 × 2 B = 77,194,752 B ≈ 77.2 MB（10⁶ B = 1 MB として）
```

解答

(1) **38,597,376 ≈ 38.6 M**
(2) **77,194,752 ≈ 77.2 M**
(3) **約 77.2 MB**

★ GPT-2 small 全体（約 124 M）の約 3 割が埋め込み行列。語彙を増やすと効いてくる。

例題 01-3　文字数・単語数・トークン数の換算

これは何？ 何に使う？

何か　LLM が数える単位「トークン」と、人間が数える文字数・単語数との換算の目安。

何のため　文章がコンテキスト長に収まるか、API 料金や応答時間がどれくらいかを事前に見積もるため。料金も上限もすべてトークン単位で決まる。

問題

英語では経験則として **1 トークン ≈ 4 文字 ≈ 0.75 単語** と見積もれる。

(1) 3,000 単語の英文レポートは約何トークンか。

(2) コンテキスト長 128,000 トークンのモデルに、このレポートは何本まで入るか。

(3) 入力料金が 100 万トークンあたり $3 のとき、1 本を送るといくらかかるか。

解説

(1) 単語 → トークン

1 トークン ≈ 0.75 単語なので、トークン数 = 単語数 ÷ 0.75。

```
3,000 ÷ 0.75 = 4,000 トークン
```

（参考: 1 単語 ≈ 5.3 文字なので 16,000 文字 ÷ 4 = 4,000 とも一致する）

(2) コンテキストに何本入るか

```
128,000 ÷ 4,000 = 32 本
```

実際には指示文や出力のぶんの余白も必要なので、少し減る。

(3) 料金

```
4,000 × ($3 ÷ 1,000,000) = $0.012
```

解答

(1) **約 4,000 トークン**　(2) **約 32 本**　(3) **約 $0.012（1.2 セント）**

★ 日本語は 1 文字 ≈ 1〜2 トークンになりやすく、英語より割高。
料金・速度・コンテキスト長はすべて「文字数」ではなく「トークン数」で決まる。

参考: github.com/amitshekhariitbhu/llm-internals

例題 02-1　X·W で Q, K, V を求める

これは何？ 何に使う？

何か　Attention の最初の処理。各トークンのベクトルを、Query（何を探すか）・Key（自分は何か）・Value（渡す中身）の 3 つに線形変換する。

何のため　「どのトークンを参照するか（Q と K）」と「参照したら何を受け取るか（V）」を別々に学習できるようにするため。

問題

3 トークンの入力 X（各行が 1 トークン、d = 2）と重み行列が次のように与えられている。

X =

1	0
0	1
1	1

W_Q =

1	1
0	1

W_K =

1	0
1	1

W_V =

1	2
0	1

Q = X·W_Q、K = X·W_K、V = X·W_V を計算せよ。

解説

行列積は「行 × 列」の内積

結果の (i, j) 成分 = X の i 行目と W の j 列目の内積。

X の各行（トークン）に同じ W を掛けて、Q・K・V の 3 つの役割に変換する。

Q = X·W_Q（W_Q の列は $(1,0)^T$ と $(1,1)^T$）

```
1行目 (1,0): (1·1+0·0, 1·1+0·1) = (1, 1)
2行目 (0,1): (0·1+1·0, 0·1+1·1) = (0, 1)
3行目 (1,1): (1+0, 1+1)         = (1, 2)
```

K と V も同じ手順

```
K: (1,0)→(1, 0)  (0,1)→(1, 1)  (1,1)→(2, 1)
V: (1,0)→(1, 2)  (0,1)→(0, 1)  (1,1)→(1, 3)
```

X の 3 行目 = 1 行目 + 2 行目 なので、結果の 3 行目も 1 行目 + 2 行目になる（検算に使える）。

解答

Q =

1	1
0	1
1	2

K =

1	0
1	1
2	1

V =

1	2
0	1
1	3

★ Q は「何を探すか」、K は「何を持っているか」、V は「渡す中身」。
同じ入力 X から、学習した 3 つの重みで別々の表現を作る。

参考: github.com/amitshekhariitbhu/llm-internals

例題 02-2　スコア $Q \cdot K^T$ を計算する

これは何？ 何に使う？

何か　各トークンの Query と全トークンの Key の内積を並べた「関連度の表」。

何のため　文中のどの単語がどの単語と関係が深いか（例:「それ」が指す名詞）を数値で表すため。値が大きいほど強く参照する。

問題

3 トークンの入力 X（各行が 1 トークン、d = 2）と重み行列が次のように与えられている。

X =

1	0
0	1
1	1

W_Q =

1	1
0	1

W_K =

1	0
1	1

W_V =

1	2
0	1

例題 02-1 で求めた Q, K を使う。

Q =

1	1
0	1
1	2

K =

1	0
1	1
2	1

(1) スコア行列 $S = Q \cdot K^T$ を求めよ。

(2) $d_k = 2$ として $S / \sqrt{d_k}$ を小数第 3 位まで求めよ。

解説

(1) S の (i, j) 成分 = トークン i の q とトークン j の k の内積

```
q1=(1,1): k1=(1,0)→1   k2=(1,1)→2   k3=(2,1)→3
q2=(0,1): k1→0         k2→1         k3→1
q3=(1,2): k1→1         k2→1+2=3     k3→2+2=4
```

値が大きいほど「トークン i がトークン j に注目したい」度合いが強い。

(2) $\sqrt{2} \approx 1.4142$ で割る

```
1/√2 = 0.707   2/√2 = 1.414   3/√2 = 2.121   4/√2 = 2.828
```

d_k が大きいと内積が大きくなりすぎるので、$\sqrt{d_k}$ で割って大きさを揃える（第 3 章）。

解答

S =

1	2	3
0	1	1
1	3	4

$S/\sqrt{2} \approx$

0.707	1.414	2.121
0.000	0.707	0.707
0.707	2.121	2.828

★ S は「トークン数 × トークン数」の正方行列。長さ n の文なら n^2 個の内積が必要。

例題 02-3　softmax で重みを出し、出力を求める

これは何？ 何に使う？

何か　スコアを softmax で「合計 1 の重み」に変え、Value の加重平均を取る Attention の最後の処理。

何のため　各トークンの表現に、文脈の中で関係の深いトークンの情報を混ぜ込み、「文脈を踏まえた意味」を持つベクトルを作るため。

問題

例題 02-2 のスケーリング後のスコアと、02-1 の V を使う。

$S/\sqrt{2}$ =

0.707	1.414	2.121
0.000	0.707	0.707
0.707	2.121	2.828

V =

1	2
0	1
1	3

(1) 各行に softmax を適用して重み行列 A を求めよ（小数第 3 位まで）。

(2) 出力 Z = A·V を求めよ。

解説

(1) softmax(z)_j = e^{z_j} / Σ_k e^{z_k}（行ごとに計算）

```
1行目: e^0.707=2.028  e^1.414=4.113  e^2.121=8.342  合計 14.483
       → 2.028/14.483=0.140  4.113/14.483=0.284  8.342/14.483=0.576
2行目: e^0=1  e^0.707=2.028  e^0.707=2.028  合計 5.056
       → 0.198  0.401  0.401
3行目: 2.028  8.342  16.918  合計 27.288 → 0.074  0.306  0.620
```

各行の和は 1 になる（確率分布）。

(2) 出力は V の行を重み付きで足したもの

```
z1 = 0.140·(1,2) + 0.284·(0,1) + 0.576·(1,3)
   = (0.140+0+0.576, 0.280+0.284+1.728) = (0.716, 2.292)
z2 = 0.198·(1,2) + 0.401·(0,1) + 0.401·(1,3) = (0.599, 2.000)
z3 = 0.074·(1,2) + 0.306·(0,1) + 0.620·(1,3) = (0.694, 2.314)
```

解答

A ≈

0.140	0.284	0.576
0.198	0.401	0.401
0.074	0.306	0.620

Z = A·V ≈

0.716	2.292
0.599	2.000
0.694	2.314

まとめると **$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}(Q \cdot K^{\mathsf{T}} / \sqrt{d_k}) \cdot V$**。

★ 出力の各行は「全トークンの V の加重平均」。どのトークンをどれだけ混ぜるかを A が決める。

参考: github.com/amitshekhariitbhu/llm-internals

例題 03-1　内積の分散が d_k になることを確かめる

これは何？ 何に使う？

何か　Attention のスコアを √d_k で割る「スケーリング」の理論的な根拠。

何のため　ベクトルの次元 d_k を大きくしても、スコアの散らばりが一定になるようにするため。モデルを大きくしても同じ設定で安定して学習できる。

問題

q と k は d_k 次元のベクトルで、各成分は互いに独立、平均 0・分散 1 とする。

(1) 内積 $q \cdot k = \Sigma\, q_i k_i$ の平均と分散を d_k で表せ。

(2) d_k = 64 と d_k = 512 のとき、q·k の標準偏差はいくつか。

(3) $q \cdot k / \sqrt{d_k}$ の標準偏差はいくつになるか。

解説

(1) 1 項 $q_i k_i$ の平均と分散

q_i と k_i は独立なので、期待値は積に分けられる。

$$E[q_i k_i] = E[q_i] \cdot E[k_i] = 0 \cdot 0 = 0$$

$$Var(q_i k_i) = E[q_i^2 k_i^2] - 0^2 = E[q_i^2] \cdot E[k_i^2] = 1 \cdot 1 = 1$$

（平均 0 なので $E[q_i^2] = Var(q_i) = 1$）

独立な d_k 個の和 → 平均も分散も足し算

$$E[q \cdot k] = 0 \qquad Var(q \cdot k) = 1 + 1 + \dots + 1 = d_k$$

(2) 標準偏差 = √分散

$$d_k = 64 : \sqrt{64} = 8 \qquad d_k = 512 : \sqrt{512} \approx 22.6$$

スコアが ±20 以上に散らばると、softmax がほぼ one-hot になってしまう（03-2）。

(3) 定数 c で割ると分散は $1/c^2$ 倍

$$Var(q \cdot k / \sqrt{d_k}) = d_k / (\sqrt{d_k})^2 = d_k / d_k = 1$$

解答

(1) **平均 0、分散 d_k**

(2) d_k = 64 → **8**、d_k = 512 → **約 22.6**

(3) d_k によらず **1**

★ √d_k で割るのは「次元を増やしてもスコアの散らばりを一定に保つ」ため。

例題 03-2　スケーリングの有無で softmax を比べる

これは何？ 何に使う？

何か　スケーリングしたときとしないときで、softmax の出力と勾配がどう変わるかの比較。

何のため　softmax が飽和（ほぼ 0 と 1 だけ）になるのを防ぎ、勾配を保って学習を安定させるため。

問題

$d_k = 16$ で、あるクエリのスコア（$Q \cdot K^T$ の 1 行）が次のようになった。

```
s = (8, 4, 0)
```

(1) スケーリングなしで softmax(s) を求めよ。

(2) $\sqrt{d_k}$ で割ってから softmax($s / \sqrt{d_k}$) を求めよ。

(3) 最大の重み p について、softmax の勾配の大きさ $p(1-p)$ を比べよ。

解説

(1) そのまま softmax

```
e⁸ = 2981.0  e⁴ = 54.6  e⁰ = 1    合計 3036.6
→ (2981.0, 54.6, 1) / 3036.6 = (0.982, 0.018, 0.0003)
```

ほぼ 1 つのトークンだけに注目する（ほぼ one-hot）。

(2) $\sqrt{16} = 4$ で割ってから softmax

```
s/4 = (2, 1, 0)
e² = 7.389  e¹ = 2.718  e⁰ = 1    合計 11.107
→ (0.665, 0.245, 0.090)
```

(3) 勾配 $\partial p / \partial s = p(1-p)$

```
スケーリングなし: 0.982 × 0.018 ≈ 0.018
スケーリングあり: 0.665 × 0.335 ≈ 0.223   （約 12 倍）
```

softmax が飽和すると勾配がほぼ 0 になり、学習が進まなくなる。

解答

(1) **(0.982, 0.018, 0.000)**　(2) **(0.665, 0.245, 0.090)**

(3) **約 0.018 と約 0.223**。スケーリングした方が勾配が約 12 倍大きい。

★ スケーリングは「softmax を飽和させず、勾配を流し続ける」ための工夫。

例題 04-1　マスク後の softmax を計算する

これは何？ 何に使う？

何か　Attention で「自分より後ろ（未来）のトークン」を見えなくするマスク。

何のため　次のトークンを予測するときに答えを先に見る「カンニング」を防ぎ、学習時と文章生成時の条件を揃えるため。GPT 型のデコーダでは必須。

問題

デコーダの 3 トークン「私 は 猫」について、スケーリング後のスコア S が次のように出た。

S =

2	1	0
1	3	1
0	2	2

行 i はトークン i（クエリ）、列 j はトークン j（キー）を表す。

(1) Causal Mask を加えたスコア行列を書け。

(2) 各行に softmax を適用して重み行列 A を求めよ（小数第 3 位まで）。

解説

(1) 未来（j > i）のスコアを −∞ にする

生成時、トークン i はまだ出ていない i+1 以降を見てはいけない。

そこで上三角（対角より右上）に −∞ を足す。e^{−∞} = 0 なので重みが 0 になる。

M =

0	-∞	-∞
0	0	-∞
0	0	0

S + M =

2	-∞	-∞
1	3	-∞
0	2	2

(2) −∞ を除いた要素だけで softmax

```
1行目: e² だけ → (1, 0, 0)
2行目: e¹=2.718, e³=20.086 合計 22.804 → (0.119, 0.881, 0)
3行目: e⁰=1, e²=7.389, e²=7.389 合計 15.778 → (0.063, 0.468, 0.468)
```

解答

A ≈

1.000	0	0
0.119	0.881	0
0.063	0.468	0.468

A は**下三角行列**になり、各行の和は 1。

★ 1 行目の「私」は自分自身しか見られないので、重みは必ず 1 になる。
マスクを softmax の「前」に入れるのがポイント（後で 0 にすると行の和が 1 でなくなる）。

参考: github.com/amitshekhariitbhu/llm-internals

例題 04-2　マスク行列の有効要素数を数える

これは何？ 何に使う？

何か　Causal Mask のうち、実際に計算に使われる（マスクされない）要素の数と割合。

何のため　Attention の計算量やメモリを見積もったり、マスクされた部分を飛ばす最適化（Flash Attention など）の効果を見積もったりするため。

問題

系列長 n の Causal Mask を考える。

(1) n = 6 のとき、注目できる（マスクされない）要素数と、マスクされる要素数を求めよ。

(2) 一般の n について、有効要素数を n の式で表せ。

(3) n = 4096 のとき、有効要素は全体の何 % か。

解説

(1) 行ごとに数える

トークン i（1 始まり）は自分を含めて i 個のトークンを見られる。

```
1 + 2 + 3 + 4 + 5 + 6 = 21
```

```
全体 6 × 6 = 36 → マスクされる要素 = 36 − 21 = 15
```

(2) 1 から n までの和

```
1 + 2 + … + n = n(n+1)/2     （対角を含む下三角）
```

(3) 割合

```
n(n+1)/2 ÷ n² = (n+1)/(2n)
```

```
n = 4096: 4097 / 8192 = 0.50012… → 約 50.01 %
```

```
（要素数: 8,390,656 / 16,777,216）
```

解答

(1) 有効 **21**、マスク **15**　(2) **n(n+1)/2**　(3) **約 50.01 %**

★ 長い系列ではスコア行列のほぼ半分が無駄。Flash Attention などは全部が −∞ のブロックを丸ごと飛ばして計算を約半分にする。

例題 05-1　多クラスのクロスエントロピーを計算する

これは何？ 何に使う？

何か　モデルの予測確率と正解との「ずれ」を測る損失関数。

何のため　分類や次トークン予測の学習で、正解の確率が上がる方向にモデルを更新するため。LLM の事前学習の損失はこれ 1 つ。

問題

3 クラス分類（猫・犬・鳥）で、モデルの出力確率が次のようになった。

猫	犬	鳥
0.7	0.2	0.1

(1) 正解が「猫」のときの損失 L を求めよ。

(2) 正解が「鳥」のときの損失 L を求めよ。

（ln 0.7 = −0.357、ln 0.1 = −2.303 を使ってよい）

解説

クロスエントロピーの式

```
L = − Σ_c y_c · ln p_c     （y は正解が 1、それ以外が 0 の one-hot）
```

正解以外の項は y_c = 0 で消えるので、**L = −ln(正解クラスの確率)** だけが残る。

(1) 正解 = 猫（y = (1, 0, 0)）

```
L = −(1·ln 0.7 + 0·ln 0.2 + 0·ln 0.1) = −ln 0.7 = 0.357
```

(2) 正解 = 鳥（y = (0, 0, 1)）

```
L = −ln 0.1 = 2.303
```

正解に低い確率しか付けていないので、損失は約 6.5 倍に跳ね上がる。

解答

(1) **L ≈ 0.357**　(2) **L ≈ 2.303**

★ −ln p は p → 1 で 0、p → 0 で ∞。「自信満々に間違える」ほど強く罰する。

例題 05-2　二値のクロスエントロピー（BCE）

これは何？ 何に使う？

何か　答えが 2 択（Yes / No）のときのクロスエントロピー。

何のため　スパム判定や、RLHF の報酬モデル（どちらの回答が好ましいか）など、2 択の確率を学習するため。

問題

4 つのサンプルについて、正解ラベル y と「1 である確率」の予測 p が次の通り。

サンプル	A	B	C	D
y	1	0	1	0
p	0.9	0.2	0.6	0.7

各サンプルの BCE と、4 つの平均を求めよ。

（ln 0.9 = −0.105、ln 0.8 = −0.223、ln 0.6 = −0.511、ln 0.3 = −1.204）

解説

BCE の式

```
L = −[ y·ln p + (1−y)·ln(1−p) ]
```

y = 1 なら −ln p、y = 0 なら −ln(1−p) だけが残る。

サンプルごとに計算

```
A (y=1, p=0.9): −ln 0.9       = 0.105
B (y=0, p=0.2): −ln(1−0.2) = −ln 0.8 = 0.223
C (y=1, p=0.6): −ln 0.6       = 0.511
D (y=0, p=0.7): −ln(1−0.7) = −ln 0.3 = 1.204   ← 誤り（0 なのに 0.7）
```

平均

```
(0.105 + 0.223 + 0.511 + 1.204) / 4 = 2.043 / 4 ≈ 0.511
```

解答

A **0.105**、B **0.223**、C **0.511**、D **1.204**、平均 **約 0.511**

★ BCE は「クラスが 2 つのクロスエントロピー」。損失の大部分は外した D から来ている。

例題 05-3　softmax + CE の勾配 p − y

これは何？ 何に使う？

何か　softmax と クロスエントロピーを組み合わせたときの、ロジットに関する勾配。

何のため　誤差逆伝播のスタート地点になる値。「予測 − 正解」という単純な式になるので、計算が速く数値的にも安定する。

問題

ロジット $z = (2, 1, 0)$ に softmax をかけ、正解クラスを 2 番目（$y = (0, 1, 0)$）とする。

(1) 確率 $p = \mathrm{softmax}(z)$ を求めよ。

(2) 損失 $L = -\ln p_2$ を求めよ。

(3) ロジットに関する勾配 $\partial L / \partial z$ を求めよ。

解説

(1) softmax

$e^2 = 7.389$　$e^1 = 2.718$　$e^0 = 1$　合計 11.107

$p = (0.665, 0.245, 0.090)$

(2) 損失

$L = -\ln 0.245 \approx 1.408$

(3) softmax と CE を合わせた勾配は p − y になる

$L = -z_2 + \ln \Sigma_k e^{z_k}$　を z_j で微分すると

$\partial L/\partial z_j = -y_j + e^{z_j} / \Sigma_k e^{z_k} = p_j - y_j$

$\partial L/\partial z = (0.665-0, 0.245-1, 0.090-0) = (0.665, -0.755, 0.090)$

勾配降下（$z \leftarrow z - \eta \cdot \partial L/\partial z$）で、正解のロジットは上がり、ほかは下がる。

解答

(1) **$p \approx (0.665, 0.245, 0.090)$**　(2) **$L \approx 1.408$**

(3) **$\partial L/\partial z \approx (0.665, -0.755, 0.090)$**　（成分の和は 0）

★ 「予測 − 正解」という非常にシンプルな勾配になるのが、softmax + CE を組み合わせる理由。

例題 05-4　言語モデルの損失と Perplexity

これは何？ 何に使う？

何か　1 トークンあたりの平均損失を指数にした、言語モデルの評価指標。

何のため　モデルが次のトークンをどれだけ迷わず予測できるかを、「平均して何択で迷っているか」という直感的な数で比べるため。

問題

言語モデルが 4 トークンの文を生成したとき、各位置で正解トークンに付けた確率が次の通り。

位置	1	2	3	4
p(正解)	0.5	0.25	0.8	0.1

(1) 平均クロスエントロピー損失（1 トークンあたり）を求めよ。

(2) Perplexity = exp(平均損失) を求めよ。

解説

(1) 各位置の −ln p を平均する

```
−ln 0.5 = 0.693  −ln 0.25 = 1.386  −ln 0.8 = 0.223  −ln 0.1 = 2.303
```

```
平均 = (0.693 + 1.386 + 0.223 + 2.303) / 4 = 4.605 / 4 ≈ 1.151
```

(2) Perplexity

```
PPL = e^{1.151} ≈ 3.162
```

別解: PPL は確率の積の −1/N 乗（幾何平均の逆数）でもある。

```
0.5 × 0.25 × 0.8 × 0.1 = 0.01 → 0.01^{−1/4} = 10^{0.5} = √10 ≈ 3.162
```

解答

(1) **約 1.151**　(2) **PPL = $\sqrt{10}$ ≈ 3.16**

★ PPL ≈ 3.16 は「毎回およそ 3 つの候補で迷っている」くらいの意味。
一様に V 語から選ぶモデルなら PPL = V になる。

例題 06-1　連鎖律で勾配を求める

これは何？ 何に使う？

何か　出力の損失から、各重みの勾配を連鎖律で効率よく求めるアルゴリズム。

何のため　何十億ものパラメータそれぞれについて「どちらにどれだけ動かせば損失が減るか」を、逆向きの計算１回でまとめて求めるため。

問題

次の小さなネットワークを考える（バイアスなし）。

$h = \mathrm{ReLU}(w_1 \cdot x) \quad \hat{y} = w_2 \cdot h \quad L = \frac{1}{2}(\hat{y} - t)^2$

入力 $x = 2$、目標 $t = 1$、初期重み $w_1 = 0.5$、$w_2 = 1.5$ とする。

(1) 順伝播で h、$\hat{y}$、L を求めよ。

(2) 連鎖律を使って $\partial L/\partial w_2$ と $\partial L/\partial w_1$ を求めよ。

解説

(1) 順伝播（左から右へ）

$$w_1 \cdot x = 0.5 \times 2 = 1 \rightarrow h = \mathrm{ReLU}(1) = 1$$

$$\hat{y} = w_2 \cdot h = 1.5 \times 1 = 1.5$$

$$L = \frac{1}{2}(1.5 - 1)^2 = \frac{1}{2} \times 0.25 = 0.125$$

(2) 逆伝播（右から左へ、局所的な微分を掛けていく）

$$\partial L/\partial \hat{y} = \hat{y} - t = 0.5$$

$$\partial L/\partial w_2 = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial w_2 = 0.5 \times h = 0.5 \times 1 = 0.5$$

$$\partial L/\partial h = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial h = 0.5 \times w_2 = 0.5 \times 1.5 = 0.75$$

ReLU の微分は「入力 > 0 なら 1、それ以外は 0」。今は入力 $1 > 0$ なので 1。

$$\partial L/\partial w_1 = \partial L/\partial h \cdot \mathrm{ReLU}'(w_1 x) \cdot x = 0.75 \times 1 \times 2 = 1.5$$

解答

(1) **$h = 1$、$\hat{y} = 1.5$、$L = 0.125$**

(2) **$\partial L/\partial w_2 = 0.5$、$\partial L/\partial w_1 = 1.5$**

★ 逆伝播は「出力側の勾配 × その場の局所微分」を入力側へ順に渡していくだけ。
もし $w_1 \cdot x \le 0$ なら $\mathrm{ReLU}' = 0$ となり、w_1 に勾配が流れない（dead ReLU）。

例題 06-2　勾配降下で重みを更新する

これは何？ 何に使う？

何か　勾配と逆向きに、重みを少しずつ動かす最適化の方法。

何のため　損失が減る方向に重みを繰り返し更新して、モデルを学習させるため。
LLM で使う Adam / AdamW もこの考え方を発展させたもの。

問題

次の小さなネットワークを考える（バイアスなし）。

$h = \mathrm{ReLU}(w_1 \cdot x) \quad \hat{y} = w_2 \cdot h \quad L = \frac{1}{2}(\hat{y} - t)^2$

入力 $x = 2$、目標 $t = 1$、初期重み $w_1 = 0.5$、$w_2 = 1.5$ とする。

例題 06-1 の勾配 $\partial L/\partial w_1 = 1.5$、$\partial L/\partial w_2 = 0.5$ を使う。

(1) 学習率 $\eta = 0.1$ で 1 ステップ更新した後の w_1, w_2 を求めよ。

(2) 更新後の重みで再び順伝播し、損失 L がどう変わったか確かめよ。

解説

(1) 更新式 $w \leftarrow w - \eta \cdot \partial L/\partial w$

$$w_1 = 0.5 - 0.1 \times 1.5 = 0.5 - 0.15 = 0.35$$

$$w_2 = 1.5 - 0.1 \times 0.5 = 1.5 - 0.05 = 1.45$$

勾配が正 → 重みを減らすと損失が下がる、という向きに動く。

(2) 新しい重みで順伝播

$$h = \mathrm{ReLU}(0.35 \times 2) = 0.7$$

$$\hat{y} = 1.45 \times 0.7 = 1.015$$

$$L = \frac{1}{2}(1.015 - 1)^2 = \frac{1}{2} \times 0.000225 = 0.0001125$$

解答

(1) **$w_1 = 0.35$、$w_2 = 1.45$**

(2) **L: 0.125 → 約 0.00011**（予測 1.5 → 1.015 で目標 1 にほぼ一致）

★ 学習率が大きすぎると行き過ぎて発散し、小さすぎると収束が遅い。
LLM の学習もこの「順伝播 → 逆伝播 → 更新」を何兆トークン分も繰り返している。